```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<title>Find My Ball — Vue reconstruction</title>
<script src="https://cdnjs.cloudflare.com/ajax/libs/vue/3.4.21/vue.global.prod.min.js"></script>
<style>
 :root{
   --bg:#f4f5f8;
   --card:#ffffff;
   --ink:#20222b;
   --sub:#8a8f9c;
   --red:#e8261c;
   --red-dark:#a3160f;
   --line:#ececf1;
 }
 *{ box-sizing:border-box; }
 html,body{ margin:0; padding:0; }
 body{
   font-family:'Segoe UI',system-ui,-apple-system,Helvetica,Arial,sans-serif;
   background:#e7e9ee;
   min-height:100vh;
   display:flex; align-items:center; justify-content:center;
   padding:24px;
 }

 .frame{
   width:380px;
   background:var(--bg);
   border-radius:22px;
   box-shadow:0 30px 70px rgba(20,20,30,.25);
   padding:22px 20px 26px;
 }

 h1{
   text-align:center;
   color:var(--red);
   font-size:24px;
   margin:2px 0 16px;
   letter-spacing:.3px;
 }

 .scoreboard{
   display:flex; align-items:center; justify-content:space-between;
```

```
    background:var(--card); border-radius:14px; padding:10px 14px; margin-bottom:14px;
    box-shadow:0 2px 8px rgba(20,20,30,.05);
  }
  .player{
    flex:1; text-align:center; padding:6px 4px; border-radius:10px;
    transition:background .3s ease, color .3s ease;
  }
  .player.turn{ background:var(--red); color:#fff; }
  .player .name{ font-size:12px; font-weight:700; text-transform:uppercase; letter-spacing:.4px;
opacity:.85; }
  .player .score{ font-size:20px; font-weight:800; margin-top:2px; }
  .vs{ font-size:11px; color:var(--sub); font-weight:700; padding:0 8px; }

  .panel{
    background:var(--card); border-radius:16px; padding:16px 14px 18px;
    box-shadow:0 2px 8px rgba(20,20,30,.05);
  }

  .status{ text-align:center; font-size:14px; font-weight:700; color:var(--ink); margin-bottom:2px; }
  .sub-status{ text-align:center; font-size:12px; color:var(--sub); margin-bottom:14px; }

  .tries{
    display:flex; justify-content:center; gap:6px; margin-bottom:14px;
  }
  .tries .pip{
    width:10px;height:10px;border-radius:50%; background:var(--line);
    transition:background .25s ease, transform .25s ease;
  }
  .tries .pip.used{ background:var(--red-dark); transform:scale(.85); }

  .grid{
    display:grid; grid-template-columns:repeat(5,1fr); gap:14px 10px;
    justify-items:center; margin-bottom:16px;
  }

  .cup-wrap{
    width:52px; height:64px; position:relative; cursor:pointer;
    display:flex; align-items:flex-end; justify-content:center;
  }
  .cup-wrap.disabled{ cursor:default; }

  .ball{
    position:absolute; bottom:2px; left:50%; transform:translateX(-50%);
    width:16px; height:16px; border-radius:50%;
```

```
  background:radial-gradient(circle at 35% 30%, #fff6c9, #f4c53a 55%, #c98f10 100%);
  box-shadow:0 2px 4px rgba(0,0,0,.3);
  opacity:0; transition:opacity .2s ease .1s;
  z-index:1;
}
.cup-wrap.reveal-ball .ball{ opacity:1; }

.cup{
  width:46px; height:56px;
  clip-path:polygon(14% 0%, 86% 0%, 100% 100%, 0% 100%);
  background:linear-gradient(180deg,#ff4c40 0%, var(--red) 55%, var(--red-dark) 100%);
  position:relative;
  transform-origin:bottom center;
  transition:transform .35s cubic-bezier(.34,1.56,.64,1), opacity .3s ease;
  z-index:2;
  box-shadow:inset 0 -6px 10px rgba(0,0,0,.15);
}
.cup::before{
  content:''; position:absolute; top:-3px; left:8%; right:8%; height:8px;
  background:#ff6b60; border-radius:50%;
}
.cup-wrap:not(.disabled):hover .cup{ transform:translateY(-3px); }

.cup-wrap.crushed .cup{
  transform:scaleY(.18) translateY(4px);
  box-shadow:inset 0 -2px 6px rgba(0,0,0,.25);
}
.cup-wrap.crushed.has-ball .cup{
  background:linear-gradient(180deg,#ff4c40 0%, var(--red) 55%, var(--red-dark) 100%);
}
.cup-wrap.crushed.empty-cup .cup{ opacity:.55; }

.cta{
  width:100%; border:none; padding:14px; border-radius:14px;
  background:var(--red); color:#fff; font-size:14px; font-weight:800;
  letter-spacing:.3px; cursor:pointer; transition:transform .15s ease, background .25s ease;
}
.cta:active{ transform:scale(.97); }
.cta.secondary{ background:var(--ink); }
.cta:disabled{ background:#d8dade; cursor:not-allowed; }

.hide-note{
  text-align:center; font-size:11px; color:var(--sub); margin-top:10px; line-height:1.5;
}
```

```
  .setup-panel{
    background:var(--card); border-radius:16px; padding:22px 18px;
    box-shadow:0 2px 8px rgba(20,20,30,.05);
  }
  .setup-panel p{
    text-align:center; font-size:12px; color:var(--sub); margin:0 0 18px;
  }
  .field{ margin-bottom:14px; }
  .field label{
    display:block; font-size:11px; font-weight:700; color:var(--sub);
    text-transform:uppercase; letter-spacing:.4px; margin-bottom:6px;
  }
  .field input{
    width:100%; padding:12px 14px; border-radius:12px; border:1.5px solid var(--line);
    font-size:14px; font-weight:600; color:var(--ink); background:var(--bg);
    outline:none; transition:border-color .2s ease;
  }
  .field input:focus{ border-color:var(--red); }
</style>
</head>
<body>
<div id="app">
  <div class="frame">
    <h1>Find My Ball</h1>

    <div v-if="phase === 'setup'" class="setup-panel">
      <p>Enter both players' names to start the game.</p>
      <div class="field">
        <label>Player 1</label>
        <input
          v-model.trim="nameInputs[0]"
          type="text"
          maxlength="16"
          placeholder="Player 1"
          @keyup.enter="startGame"
        />
      </div>
      <div class="field">
        <label>Player 2</label>
        <input
          v-model.trim="nameInputs[1]"
          type="text"
          maxlength="16"
```

```
      placeholder="Player 2"
      @keyup.enter="startGame"
    />
  </div>
  <button class="cta" @click="startGame">Start game</button>
</div>

<template v-else>
<div class="scoreboard">
  <div class="player" :class="{ turn: activeIndex === 0 }">
    <div class="name">{{ players[0].name }}</div>
    <div class="score">{{ players[0].score }}</div>
  </div>
  <div class="vs">VS</div>
  <div class="player" :class="{ turn: activeIndex === 1 }">
    <div class="name">{{ players[1].name }}</div>
    <div class="score">{{ players[1].score }}</div>
  </div>
</div>

<div class="panel">
  <div class="status">{{ statusTitle }}</div>
  <div class="sub-status">{{ statusSub }}</div>

  <div class="tries" v-if="phase === 'guessing'">
    <div class="pip" v-for="n in maxTries" :key="n" :class="{ used: n > triesLeft }"></div>
  </div>

  <div class="grid">
    <div
      v-for="(cup, i) in cups"
      :key="cup.id"
      class="cup-wrap"
      :class="{
        crushed: cup.crushed,
        'has-ball': cup.crushed && cup.hasBall,
        'empty-cup': cup.crushed && !cup.hasBall,
        'reveal-ball': cup.crushed && cup.hasBall,
        disabled: !clickable
      }"
      @click="clickCup(i)"
    >
      <div class="ball"></div>
      <div class="cup"></div>
```

```
      </div>
    </div>

    <button
      v-if="phase === 'reveal'"
      class="cta"
      @click="nextRound"
    >
      {{ players[hiderIndex].name }}'s turn to hide — Continue
    </button>

    <button
      v-else-if="phase === 'ready'"
      class="cta"
      @click="beginHiding"
    >
      {{ players[hiderIndex].name }}, tap to hide the ball
    </button>

    <div class="hide-note" v-if="phase === 'hiding'">
      Psst {{ players[hiderIndex].name }} — pick any cup below. {{ players[guesserIndex].name }}
shouldn't watch!
    </div>
    <div class="hide-note" v-else-if="phase === 'guessing'">
      {{ players[guesserIndex].name }}, tap a cup to crush it and check for the ball.
    </div>
  </div>
  </template>
 </div>
</div>

<script>
const { createApp, reactive, ref, computed } = Vue;

createApp({
 setup(){
  const NUM_CUPS = 10;
  const MAX_TRIES = 3;

  const players = reactive([
    { name: 'Player 1', score: 0 },
    { name: 'Player 2', score: 0 },
  ]);
```

```
  const nameInputs = reactive(['', '']);

  const hiderIndex = ref(0);
  const guesserIndex = computed(() => 1 - hiderIndex.value);
  const activeIndex = computed(() =>
    phase.value === 'guessing' ? guesserIndex.value : hiderIndex.value
  );

  const maxTries = MAX_TRIES;
  const triesLeft = ref(MAX_TRIES);

  // phases: 'setup' -> 'ready' -> 'hiding' -> 'guessing' -> 'reveal'
  const phase = ref('setup');

  function startGame(){
    players[0].name = nameInputs[0].trim() || 'Player 1';
    players[1].name = nameInputs[1].trim() || 'Player 2';
    players[0].score = 0;
    players[1].score = 0;
    hiderIndex.value = 0;
    phase.value = 'ready';
  }

  function freshCups(){
    return Array.from({ length: NUM_CUPS }, (_, i) => ({
      id: i, hasBall: false, crushed: false,
    }));
  }
  const cups = reactive(freshCups());

  const clickable = computed(() => phase.value === 'hiding' || phase.value === 'guessing');

  const statusTitle = computed(() => {
    if (phase.value === 'ready')   return `Get ready, ${players[hiderIndex.value].name}!`;
    if (phase.value === 'hiding')  return `${players[hiderIndex.value].name} is hiding the ball…`;
    if (phase.value === 'guessing')return `${players[guesserIndex.value].name}'s turn to guess`;
    return roundResult.message;
  });

  const statusSub = computed(() => {
    if (phase.value === 'ready')   return 'Choose a cup to secretly hide the ball under.';
    if (phase.value === 'hiding')  return 'Tap any cup to hide the ball there.';
    if (phase.value === 'guessing')return `${triesLeft.value} ${triesLeft.value === 1 ? 'try' : 'tries'}
left to find it.`;
```

```
  return roundResult.sub;
});

const roundResult = reactive({ message: '', sub: '' });

function beginHiding(){
  Object.assign(cups, freshCups());
  cups.forEach((c, i) => (cups[i].crushed = false, cups[i].hasBall = false));
  phase.value = 'hiding';
}

function clickCup(i){
  if (!clickable.value) return;

  if (phase.value === 'hiding') {
    cups[i].hasBall = true;
    triesLeft.value = MAX_TRIES;
    phase.value = 'guessing';
    return;
  }

  if (phase.value === 'guessing') {
    if (cups[i].crushed) return;
    cups[i].crushed = true;

    if (cups[i].hasBall) {
      players[guesserIndex.value].score++;
      roundResult.message = `${players[guesserIndex.value].name} found it! 🎉`;
      roundResult.sub = `+1 point for ${players[guesserIndex.value].name}.`;
      revealAll();
      phase.value = 'reveal';
    } else {
      triesLeft.value--;
      if (triesLeft.value <= 0) {
        players[hiderIndex.value].score++;
        roundResult.message = `${players[guesserIndex.value].name} couldn't find it!`;
        roundResult.sub = `+1 point for ${players[hiderIndex.value].name}. The ball was here:`;
        revealAll();
        phase.value = 'reveal';
      }
    }
  }
}
```

```
    function revealAll(){
      cups.forEach(c => { c.crushed = true; });
    }

    function nextRound(){
      hiderIndex.value = guesserIndex.value;
      phase.value = 'ready';
    }

    return {
      players, nameInputs, hiderIndex, guesserIndex, activeIndex,
      cups, triesLeft, maxTries, phase, clickable,
      statusTitle, statusSub,
      startGame, beginHiding, clickCup, nextRound,
    };
  }
}).mount('#app');
</script>
</body>
</html>
```